

Arquitetura da Solução e Padrões de Projeto

Projeto: EscolaAlerta

Dupla: Mateus Farias Bentes e Mariana Idalina Bezerra Vidinho

1. Introdução

Este documento apresenta a arquitetura de software do EscolaAlerta e os padrões de projeto (design patterns) adotados na sua implementação. O objetivo é registrar, antes da fase de desenvolvimento, como os componentes do sistema se organizam e quais soluções reutilizáveis serão aplicadas para resolver os principais problemas de design identificados a partir dos requisitos já levantados no pré-projeto.

2. Arquitetura da Solução

O EscolaAlerta segue uma arquitetura em camadas (layered architecture), com separação clara entre apresentação, aplicação e dados. Essa escolha favorece a manutenibilidade e a testabilidade do sistema, características importantes para um projeto desenvolvido por uma dupla, em que cada camada pode ser implementada e testada de forma isolada. O aplicativo Flutter consome a API por meio de requisições REST em JSON; a API concentra toda a lógica de negócio, incluindo o motor de correlação temporal que gera os alertas; e o PostgreSQL persiste os dados de alunos, indicadores e histórico de intervenções.

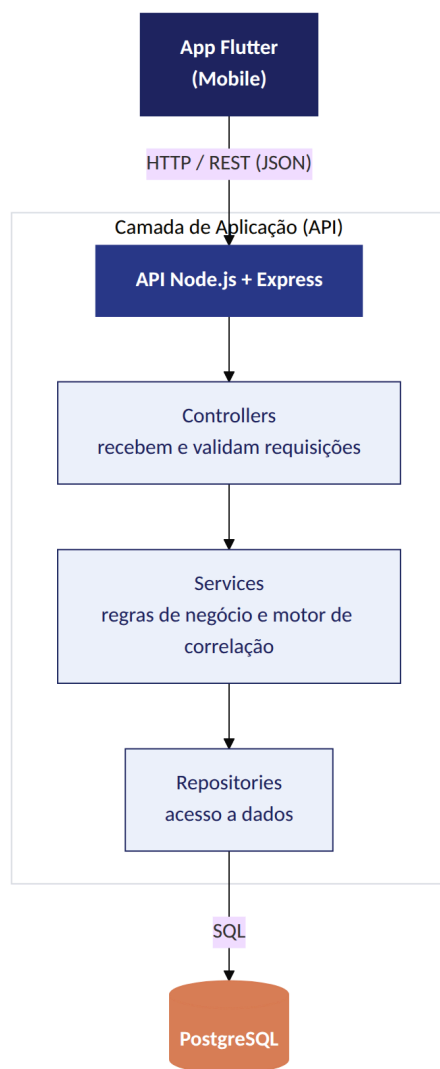


Figura 1: Arquitetura em camadas do EscolaAlerta.

Camada	Responsabilidade	Tecnologia
App Flutter (Mobile)	Interface com o usuário (pai/mãe, escola). Consome a API via REST.	Flutter/Dart
Controllers	Recebem as requisições HTTP, validam entrada e delegam para os Services.	Node.js + Express
Services	Regras de negócio: motor de correlação temporal e módulo de nudge.	Node.js
Repositories	Isolam o acesso a dados; convertem objetos de domínio em consultas SQL.	Node.js (pg)
Banco de Dados	Armazena alunos, indicadores, alertas e histórico de intervenções.	PostgreSQL

3. Padrões de Projeto (Design Patterns)

Um design pattern é uma solução reutilizável para um problema recorrente de design de software, documentada de forma independente de linguagem de programação (GAMMA et al., 1994). A seguir são justificados os três padrões escolhidos para o EscolaAlerta, cada um associado a um problema concreto identificado nos requisitos do sistema.

3.1 Strategy: Motor de Regras (Correlação Temporal)

Problema: o motor de correlação precisa avaliar diferentes tipos de indicador (notas, faltas, ocorrências) e permitir que novos critérios de risco sejam adicionados no futuro, sem exigir alterações no código já existente.

Solução aplicada: cada critério de avaliação é implementado como uma estratégia independente (RegraNotas, RegraFaltas, RegraOcorrencias), todas seguindo a mesma interface. O MotorDeRegras atua como contexto, combinando o resultado das estratégias ao longo do tempo para decidir se um alerta deve ser disparado.

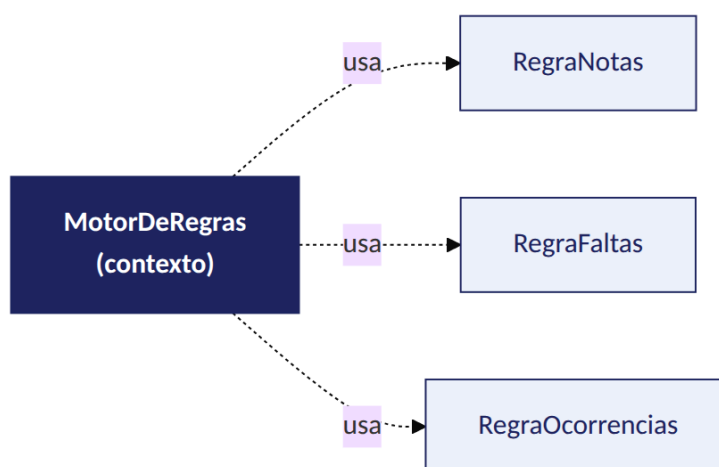


Figura 2: Strategy Pattern aplicado ao motor de regras.

Por que este padrão?: a alternativa mais simples seria uma sequência de condicionais (if/else) dentro do motor de regras. Isso funcionaria para o escopo atual, mas cada novo indicador aumentaria a complexidade de uma única função, dificultando testes e manutenção. Com Strategy, cada regra é testada

isoladamente e novos critérios são incluídos sem tocar no motor existente, o que é especialmente relevante porque o motor de correlação é o principal diferencial técnico do projeto.

3.2 Repository: Camada de Acesso a Dados

Problema: as regras de negócio não devem depender diretamente de comandos SQL, sob risco de acoplar toda a lógica do sistema ao PostgreSQL e dificultar testes automatizados.

Solução aplicada: toda operação de leitura ou escrita no banco passa por um Repository (ex.: AlunoRepository), responsável exclusivamente por converter objetos de domínio em consultas SQL e vice-versa. Os Services conhecem apenas a interface do repositório, nunca o banco em si.



Figura 3: Repository Pattern aplicado à camada de dados.

Por que este padrão?: é uma prática consolidada em arquiteturas de API REST (FOWLER, 2002) e de baixo custo de implementação. Isolar o acesso a dados permite trocar detalhes de armazenamento sem afetar as regras de negócio, e possibilita testar os Services com dados simulados (mocks), sem precisar de um banco real.

3.3 Observer: Módulo de Intervenção Orientada

Problema: quando o motor de regras dispara um alerta, mais de uma ação pode precisar acontecer (notificar a família, atualizar o painel da escola e, futuramente, enviar e-mail), sem que o motor de regras precise conhecer cada um desses destinos.

Solução aplicada: o disparo de um alerta funciona como um sujeito observável, e cada canal de resposta (notificação push, atualização de painel, envio de e-mail) é um observador independente, registrado para reagir ao evento sem se comunicar diretamente com o motor de regras.

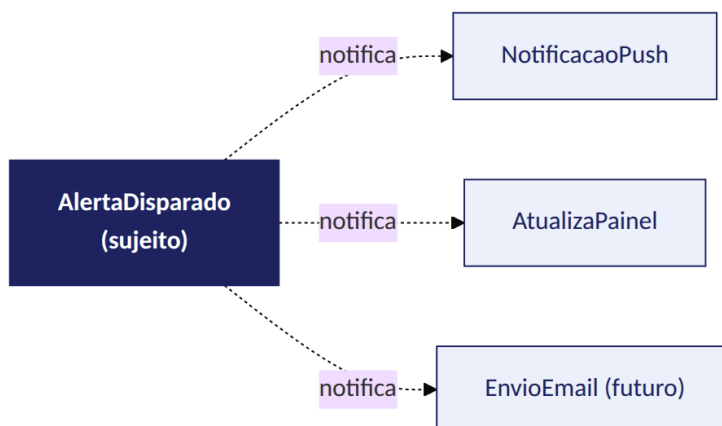


Figura 4: Observer Pattern aplicado ao módulo de nudge.

Por que este padrão?: sem esse desacoplamento, cada novo canal de notificação exigiria alterar o motor de regras diretamente. Com Observer, um novo canal é adicionado apenas registrando mais um observador, o que é importante porque o módulo de intervenção orientada é o diferencial inovador do EscolaAlerta e deve poder crescer (novos tipos de sugestão, novos canais) sem reescrever o motor de correlação.

4. Considerações Finais

A combinação de uma arquitetura em camadas com os três padrões descritos organiza o sistema em torno das suas partes mais críticas: o motor de correlação (Strategy), o acesso a dados (Repository) e a entrega das intervenções orientadas (Observer). Essa estrutura será revisada e detalhada durante a modelagem do sistema, podendo incorporar outros padrões à medida que novos requisitos forem especificados.

Referências

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley, 1994.